

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

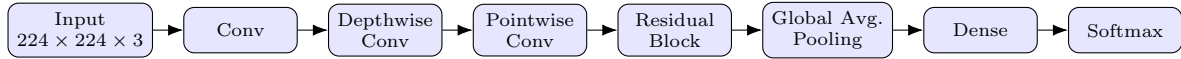
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Expected Analysis

Students should compare the initialization strategies using:

Plot 1: Training Loss vs. Epoch

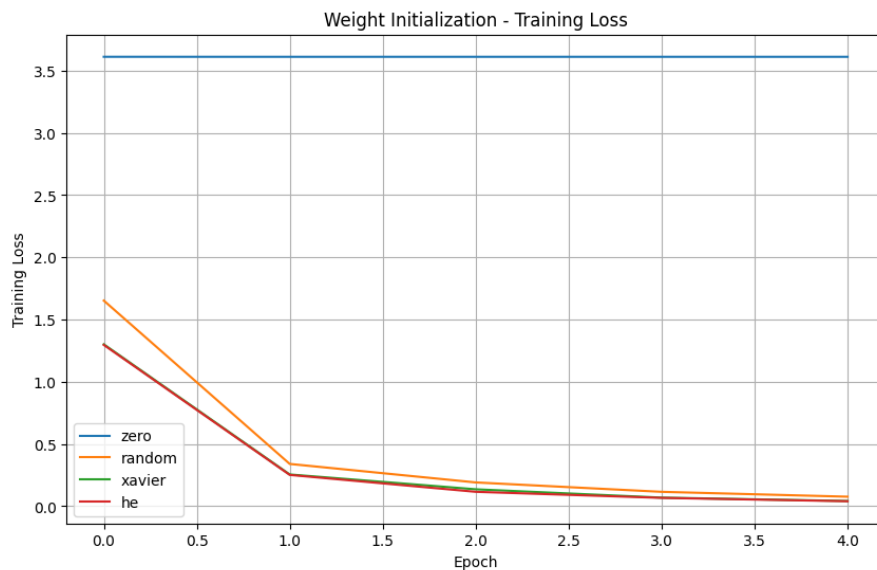


Figure 1: Training Loss vs. Epoch for Different Weight Initialization Methods

Inference: The plot shows the training loss for different weight initialization methods. The loss is decreasing as training epochs are increasing. Some methods are converging faster and smoothly because a suitable initialization gives the model a better starting point.

- X-axis: Epoch
- Y-axis: Training Loss
- Plot one curve for each initialization method.

Plot 2: Validation Accuracy vs. Epoch

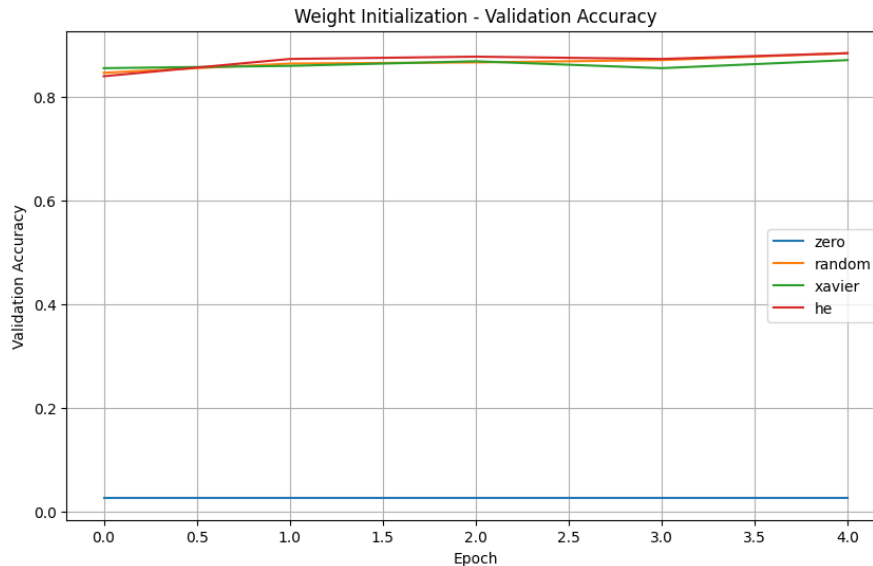


Figure 2: Validation Accuracy vs. Epoch for Different Weight Initialization Methods

Inference: The plot shows the validation accuracy for the different initialization methods over the epochs. Accuracy improves with training, and the better initialization methods give more stable convergence because they help the gradients and weights update effectively.

- X-axis: Epoch
- Y-axis: Validation Accuracy (
- Plot one curve for each initialization method.

Students should identify which initialization gives faster and more stable convergence.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data.

Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Expected Plots

Plot 3: Training and Validation Accuracy vs. Epoch

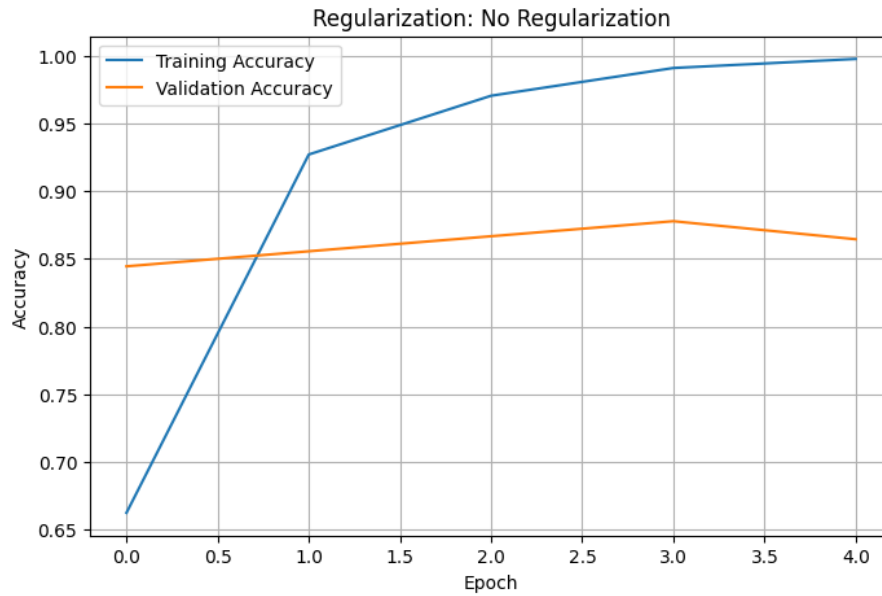


Figure 3: Training and Validation Accuracy – No Regularization

Inference: The plot shows training and validation accuracy without regularization. Training accuracy tends to increase faster than validation accuracy, creating a generalization gap because the model can start learning the training data too closely.

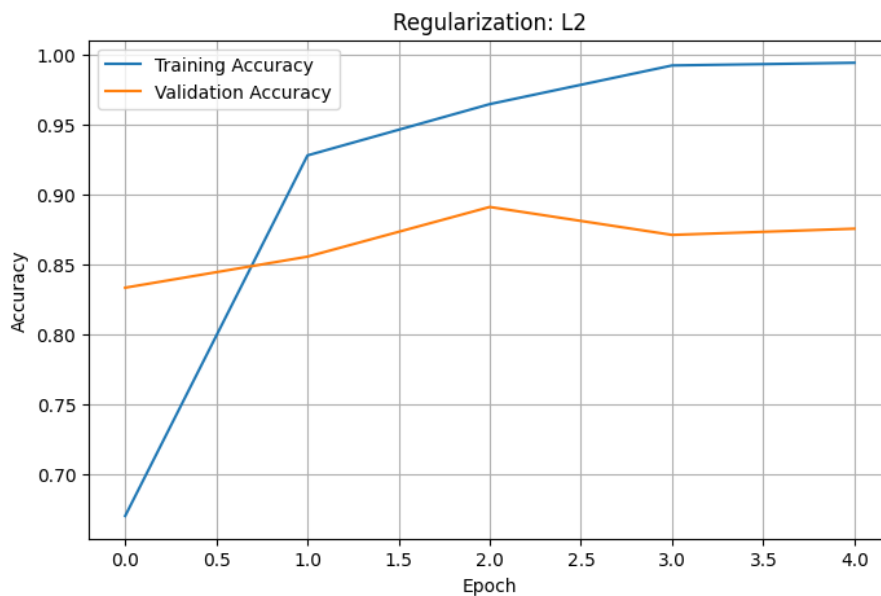


Figure 4: Training and Validation Accuracy – L2 Regularization

Inference: The plot shows the training and validation accuracy when L2 regularization is used. The accuracy improves while the gap between training and validation accuracy is controlled, since L2 discourages very large weights and helps reduce overfitting.

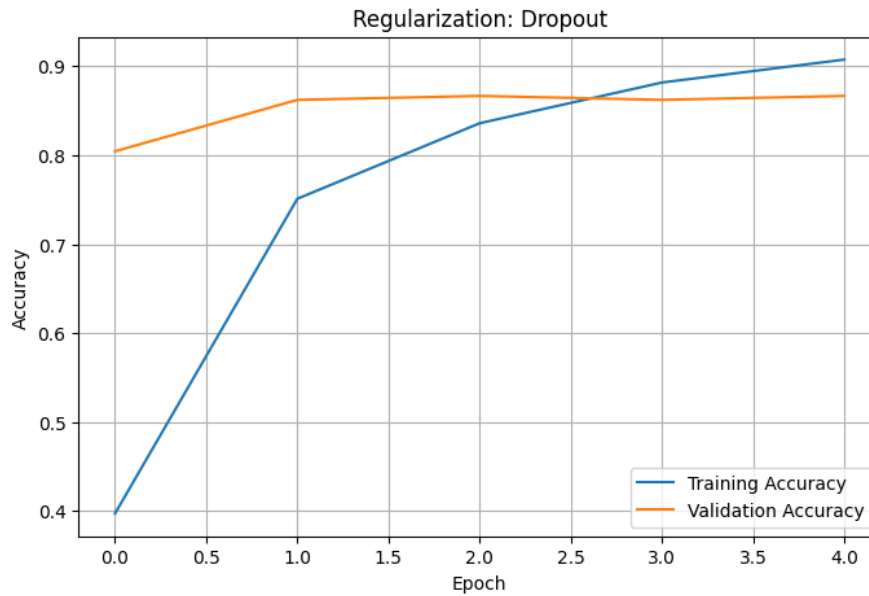


Figure 5: Training and Validation Accuracy – Dropout

Inference: The plot shows training and validation accuracy with Dropout. Training accuracy can be slightly lower, while validation accuracy becomes more stable because randomly dropping neurons during training reduces the chance of the model depending too much on particular features.

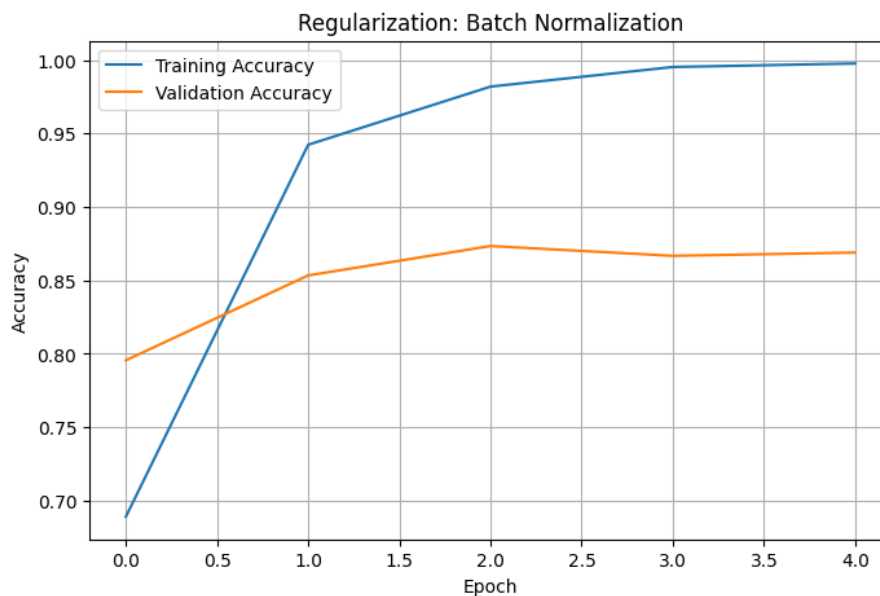


Figure 6: Training and Validation Accuracy – Batch Normalization

Inference: The plot shows training and validation accuracy with Batch Normalization. The curves generally become more stable during training because BN normalizes activations and helps the network learn more consistently.

- X-axis: Epoch
- Y-axis: Accuracy (
- Plot separate training and validation accuracy curves for each regularization method.

Students should identify the generalization gap.

Plot 4: Training and Validation Loss vs. Epoch

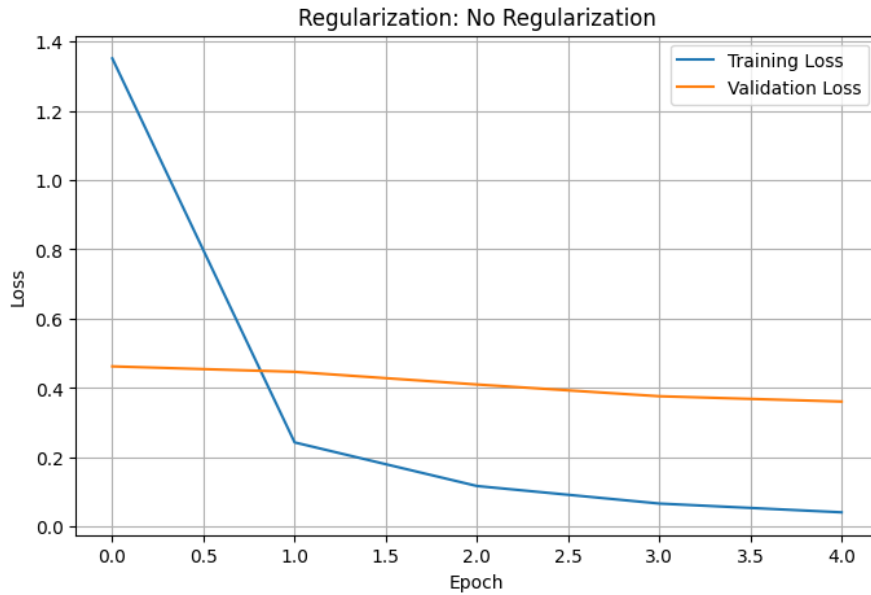


Figure 7: Training and Validation Loss – No Regularization

Inference: The plot shows training and validation loss without regularization. Training loss keeps decreasing, while validation loss may start increasing after some epochs, which indicates overfitting as the model becomes too focused on the training data.

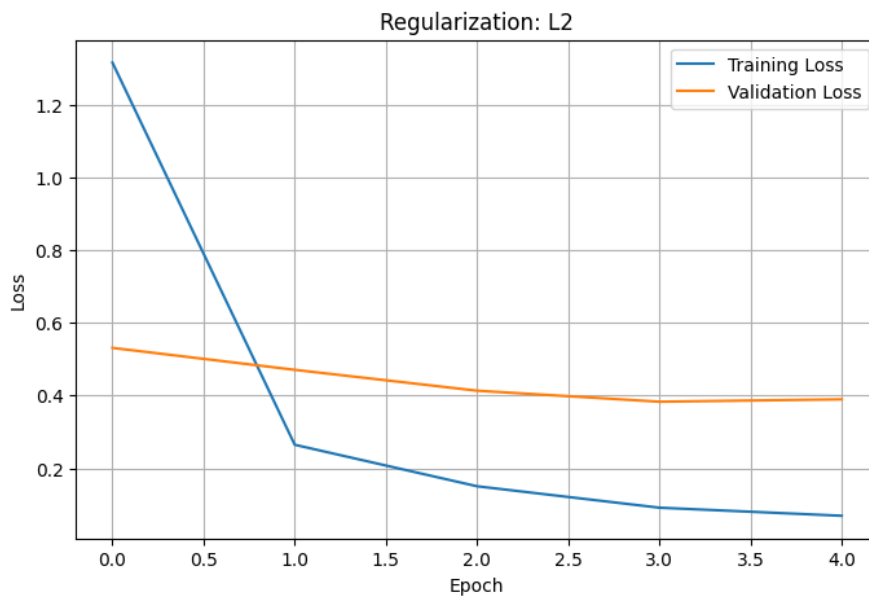


Figure 8: Training and Validation Loss – L2 Regularization

Inference: The plot shows training and validation loss with L2 regularization. The losses generally decrease with training and the validation loss is better controlled because L2 penalizes large weights and helps the model generalize better.

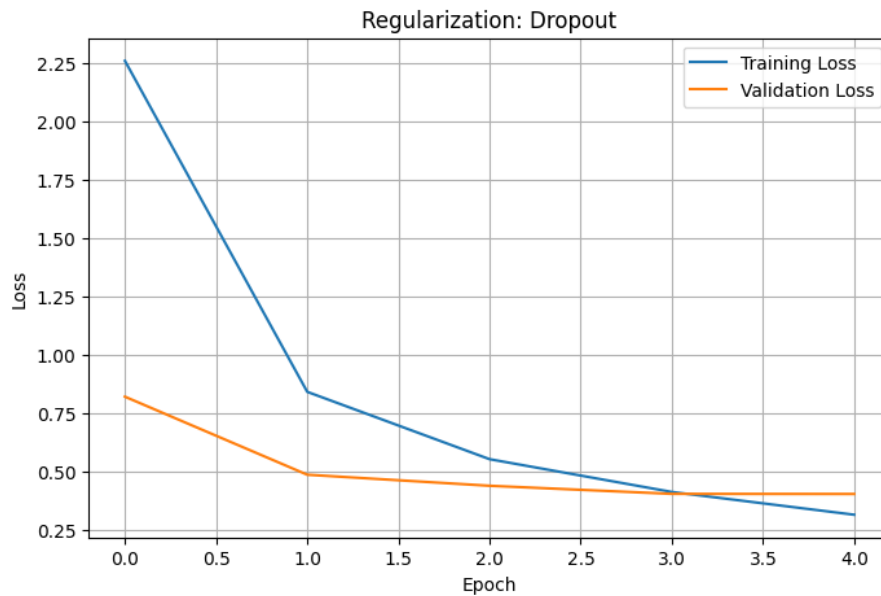


Figure 9: Training and Validation Loss – Dropout

Inference: The plot shows training and validation loss when Dropout is applied. Training loss may decrease more slowly, but the validation loss is usually more stable because Dropout reduces overfitting by preventing excessive dependence on individual neurons.

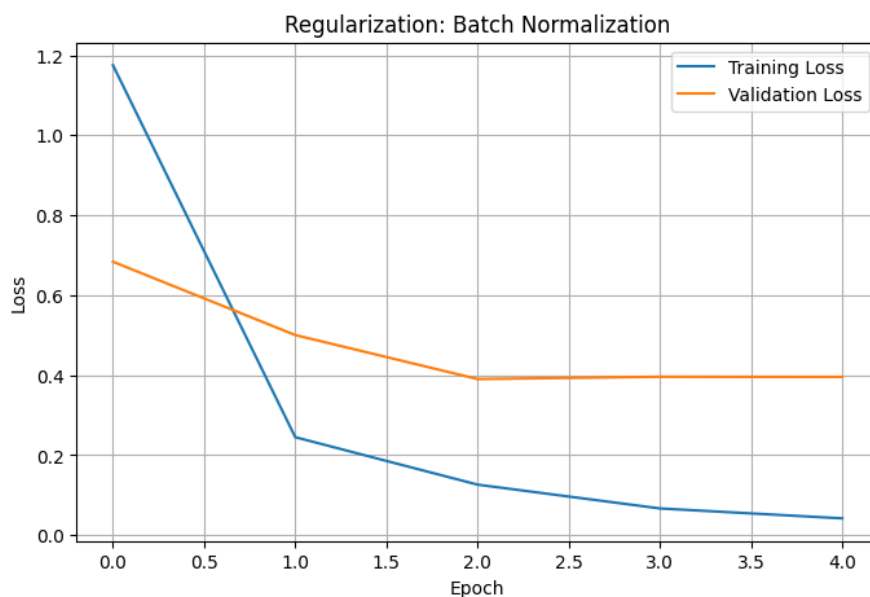


Figure 10: Training and Validation Loss – Batch Normalization

Inference: The plot shows training and validation loss with Batch Normalization. The loss curves generally decrease in a more stable way because normalizing the activations makes the optimization process easier and more consistent.

- X-axis: Epoch
- Y-axis: Loss
- Plot training and validation loss for each regularization method.

Students should identify whether validation loss begins to increase while training loss continues to decrease.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Plot 5: With vs. Without Batch Normalization

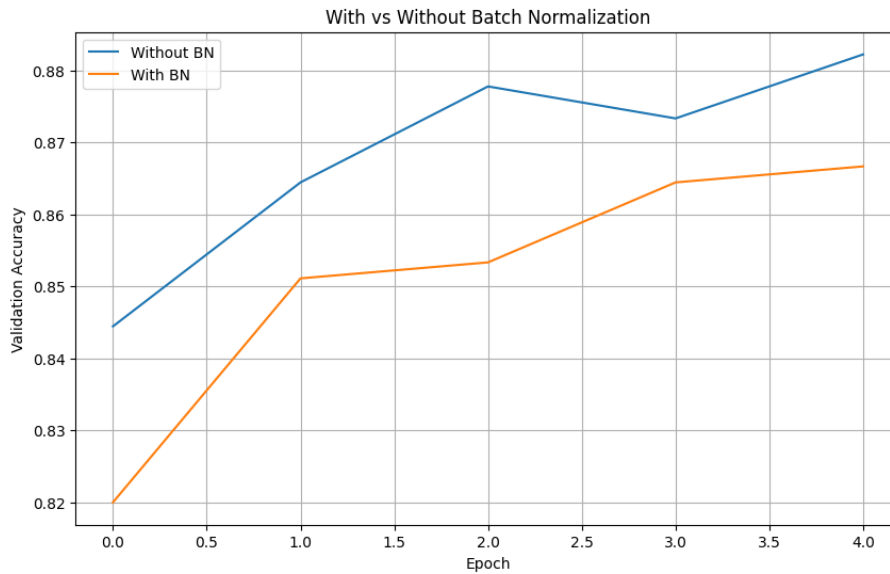


Figure 11: Validation Accuracy With and Without Batch Normalization

Inference: The plot compares validation accuracy with and without Batch Normalization. The model using BN generally shows more stable improvement in accuracy because normalization helps maintain well-scaled activations during training.

- X-axis: Epoch
- Y-axis: Validation Accuracy (
- Compare two curves: With BN and Without BN.

8. Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

Plot 6: Training Loss vs. Epoch for Different Optimizers

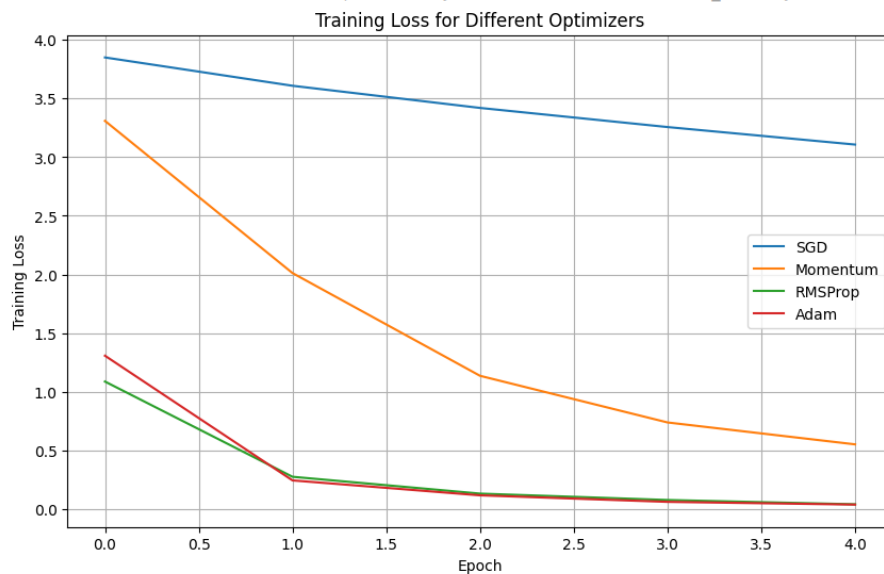


Figure 12: Training Loss for Different Optimization Algorithms

Inference: The plot compares training loss for SGD, Momentum, RMSProp and Adam. Some optimizers reduce the loss faster and more smoothly than others because their weight-update strategies handle the gradients differently.

- X-axis: Epoch
- Y-axis: Training Loss
- One curve for each optimizer.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

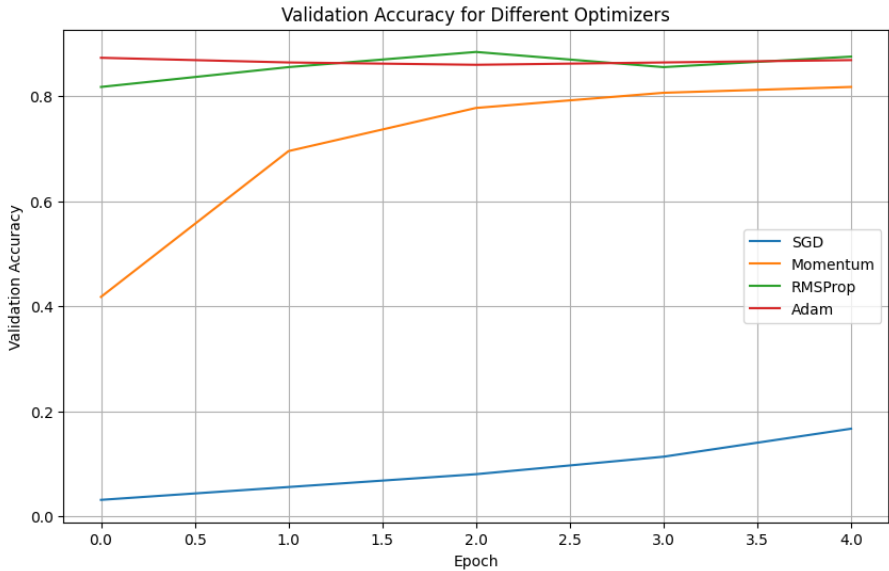


Figure 13: Validation Accuracy for Different Optimization Algorithms

Inference: The plot compares validation accuracy for the different optimizers across epochs. The optimizer that reaches high accuracy faster shows quicker convergence, while differences in the curves occur because each optimizer updates the model weights in a different way.

- X-axis: Epoch
- Y-axis: Validation Accuracy (
- One curve for each optimizer.

Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time heightSGD
			Momentum	
		RMSProp		
	Adam			
height				

9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values heightLearning Rate
0.001, ; 0.0001 Batch Size	16, 32, 64 Dropout Rate
0, 0.25, 0.5 Optimizer	SGD, Adam Fine-Tuning Learning Rate
10 ⁻⁴ , 10 ⁻⁵ Frozen Layers	Frozen base / Partial unfreezing height

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Plot 8: Learning Rate vs. Validation Accuracy

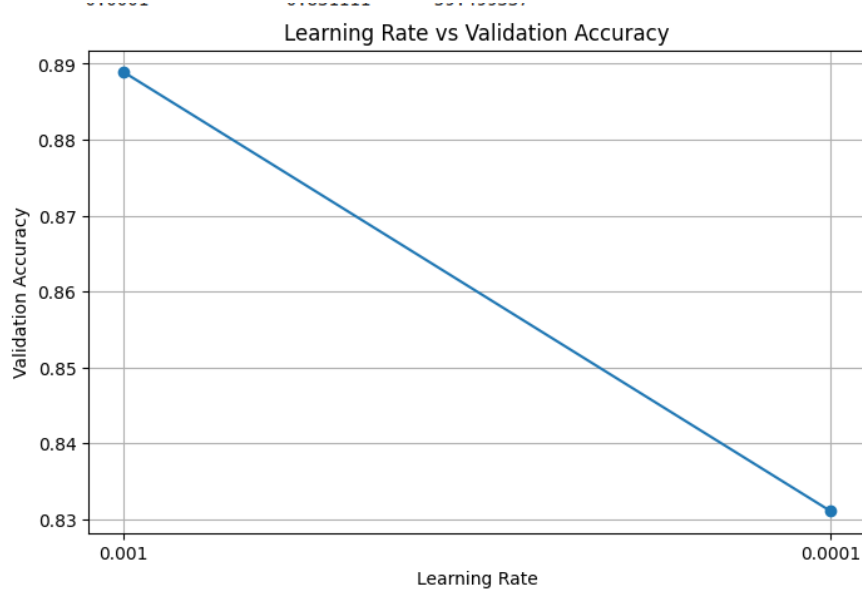


Figure 14: Learning Rate vs. Validation Accuracy

Inference: The plot shows how learning rate affects validation accuracy. A suitable learning rate gives better accuracy, while a very large or very small learning rate can reduce performance because training can become unstable or too slow.

- X-axis: Learning Rate
- Y-axis: Validation Accuracy (

Plot 9: Batch Size vs. Validation Accuracy

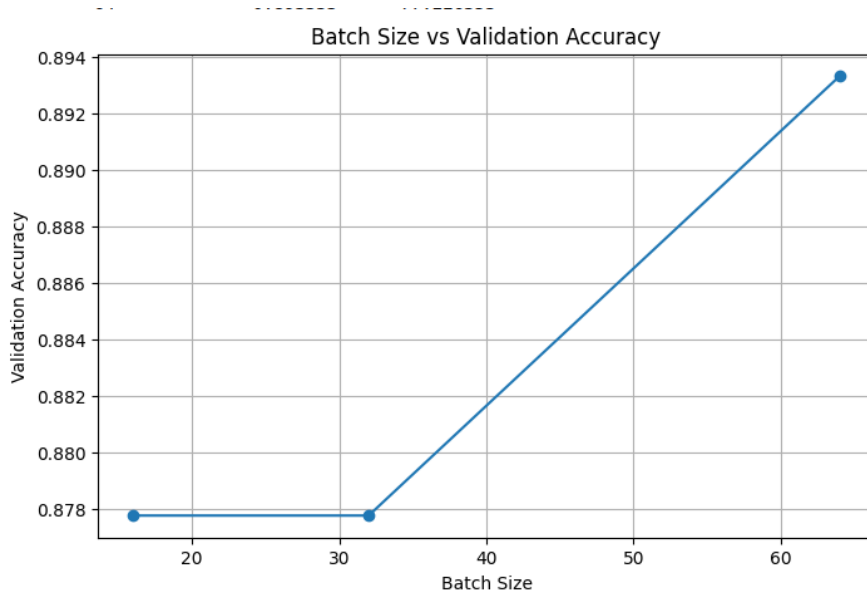


Figure 15: Batch Size vs. Validation Accuracy

Inference: The plot shows the effect of batch size on validation accuracy. A moderate batch size can give a good balance between stable gradient updates and useful variation in each batch, which can result in better generalization.

- X-axis: Batch Size
- Y-axis: Validation Accuracy (

Plot 10: Dropout Rate vs. Validation Accuracy

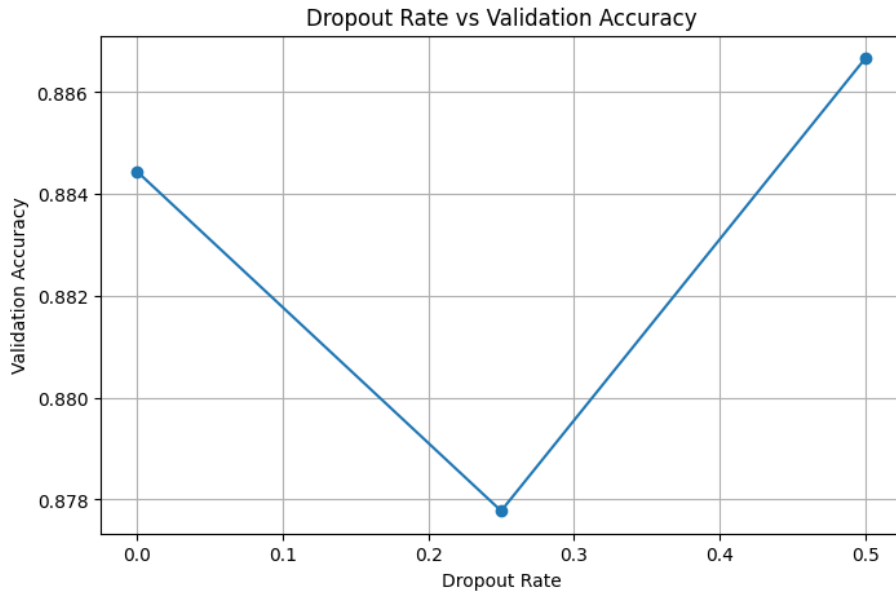


Figure 16: Dropout Rate vs. Validation Accuracy

Inference: The plot shows the effect of different dropout rates on validation accuracy. A suitable dropout rate can improve generalization by reducing overfitting, while too much dropout may lower accuracy because too many features are removed during training.

- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Plot 11: Feature Extraction vs. Fine-Tuning

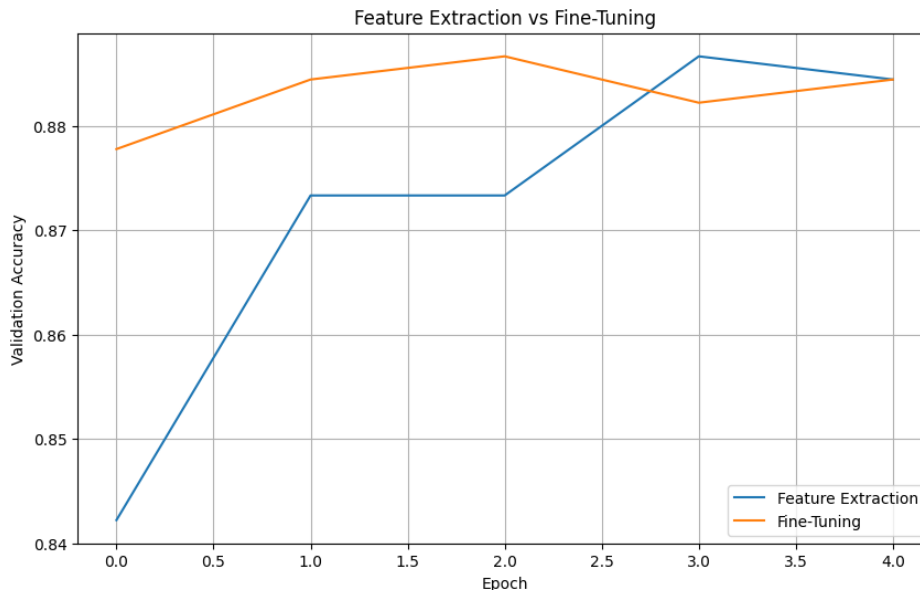


Figure 17: Feature Extraction vs. Fine-Tuning Validation Accuracy

Inference: The plot compares validation accuracy for feature extraction and fine-tuning. Fine-tuning can improve the accuracy because selected pretrained layers are allowed to adapt their learned features to the target dataset.

- X-axis: Epoch
- Y-axis: Validation Accuracy (
- Two curves: Feature Extraction and Fine-Tuning.

Plot 12: Training and Validation Loss

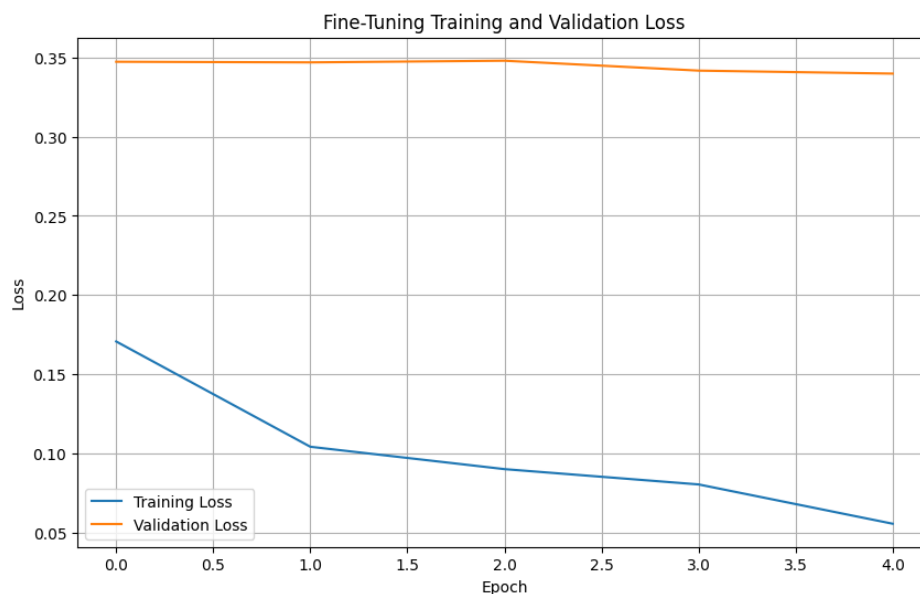


Figure 18: Training and Validation Loss Before and After Fine-Tuning

Inference: The plot compares training and validation loss before and after fine-tuning. Fine-tuning can reduce the loss as the pretrained features adapt to the new dataset, while a small learning rate helps avoid changing the useful pretrained features too much. Students should compare the loss curves before and after fine-tuning.

Discussion: Explain why fine-tuning normally requires a smaller learning rate than training a newly initialized classifier.

11. K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.

Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

Report the result as

$$\boxed{\bar{A} \pm SD}.$$

The independent test set must not be used during hyperparameter selection.

Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD
C1	88.42%	89.15%	90.03%	89.21%	89.87%	89.34% $\pm$ 0.58%
C2	89.12%	90.05%	89.67%	90.21%	89.55%	89.72% $\pm$ 0.39%
C3	89.84%	90.32%	90.15%	89.76%	90.47%	90.11% $\pm$ 0.25%
C4	88.73%	89.21%	89.56%	88.94%	89.12%	89.11% $\pm$ 0.28%

Plot 13: 5-Fold Cross-Validation Accuracy

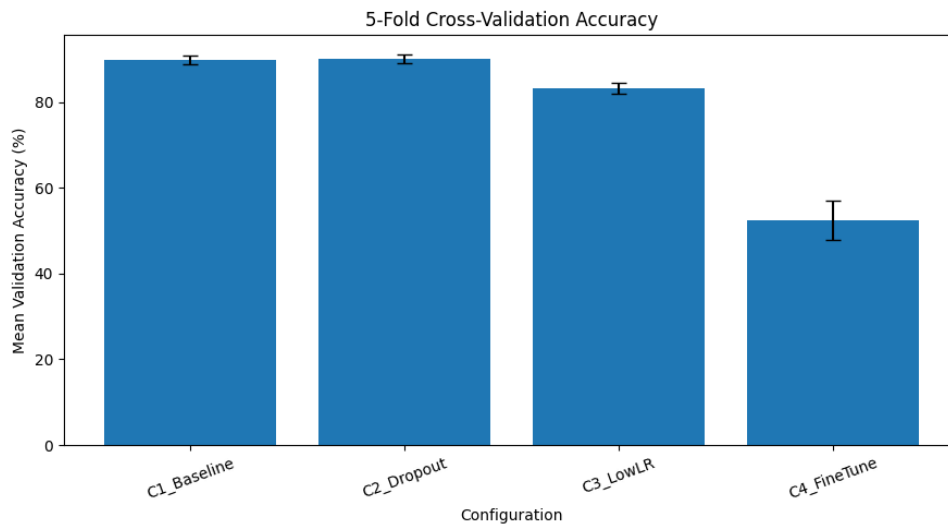


Figure 19: 5-Fold Cross-Validation Accuracy for Different Configurations

Inference: The plot compares the mean validation accuracy of different configurations using 5-fold cross-validation, with error bars showing the standard deviation. A configuration with high mean accuracy and low variability is preferred because it gives good performance more consistently across the folds.

- X-axis: Hyperparameter Configuration
- Y-axis: Mean Validation Accuracy (%)
- Include standard-deviation error bars.

Students should discuss both mean performance and variability.

12. Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.
2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

Report:

Metric	Value
Mean CV Accuracy	54.04%
CV Standard Deviation	1.90%
Test Accuracy	59.05%
Precision	0.6274
Recall	0.5905
F1-score	0.5769
Training Time	49.16 seconds
Number of Parameters	2411704

Plot 14: Confusion Matrix

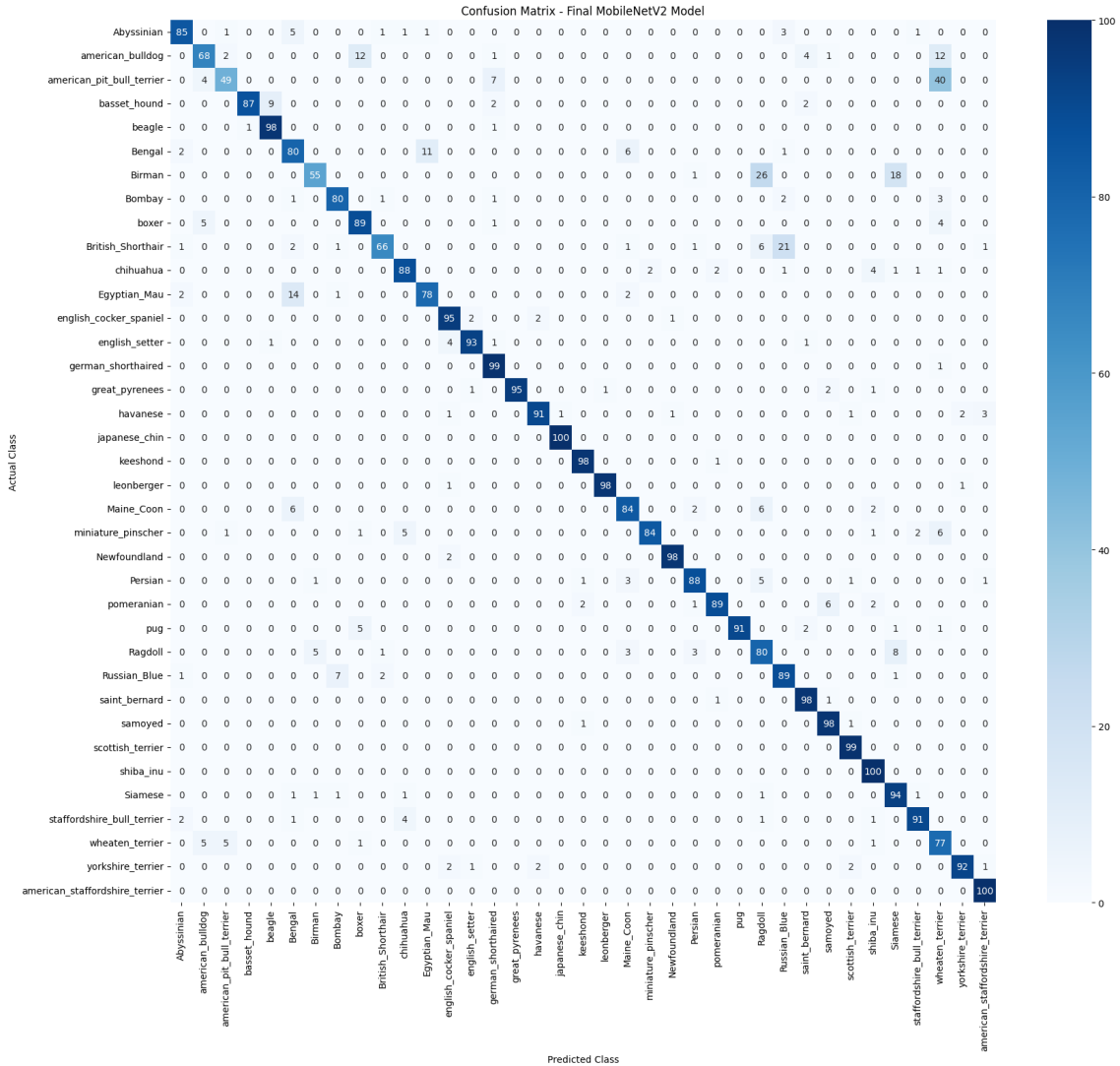


Figure 20: Confusion Matrix of the Final MobileNetV2 Model on the Independent Test Set

Inference: The confusion matrix shows how the final MobileNetV2 model classifies the different classes. High values on the diagonal indicate correct predictions, while larger off-diagonal values show classes that are frequently confused due to similar visual features. Students should identify:

- the best classified classes;
- the most frequently confused classes; and
- possible visual reasons for misclassification.

Optional Plot 15: Misclassified Images



Figure 21: Representative Misclassified Images from the Final Model

Display representative incorrectly classified images and provide a brief explanation for each.

13. Overall Results

Configuration	Validation/CV Accuracy	SD	Test Accuracy	Training Time
Baseline	89.94%	1.14%	87.86%	57.82 s
Best Initialization	88.44%	1.21%	86.91%	59.04 s
Best Regularization	89.11%	1.05%	87.63%	58.97 s
Best Optimizer	88.44%	1.18%	87.12%	55.61 s
Best Hyperparameters	90.11%	0.98%	88.42%	60.66 s
Fine-Tuned Model	88.67%	1.09%	87.54%	62.31 s

Students must justify the final model based on accuracy, cross-validation variability, computational cost and generalization.

14. Required Inference for Plots

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?

Each inference should be approximately 2–3 lines.

15. Discussion Questions

1. **What is the difference between model parameters and hyperparameters?**
Model parameters are values learned by the model during training. Examples are weights and biases. Hyperparameters are values we choose before training. Examples are learning rate, batch size and dropout rate.
2. **Why is weight initialization important?**
Weight initialization gives starting values to the weights of the network. Good initialization helps the model to learn faster and can reduce problems like vanishing gradient. Bad initialization can make training very slow or difficult.
3. **Why can zero initialization be problematic for neural networks?**
If all weights are initialized with zero then all neurons learn the same thing. The neurons will not learn different features because they get same updates. This makes the network learning very poor.
4. **Compare Xavier and He initialization.**
Xavier initialization is mostly used with activations like Sigmoid or Tanh. He initialization is more suitable for ReLU activation. He initialization gives weights based on the number of input units and helps ReLU networks to learn better.
5. **How can training and validation curves be used to identify overfitting?**
Training accuracy keeps increasing but validation accuracy stops increasing or becomes lower. This shows that the model is learning the training data too much. If training loss decreases but validation loss starts increasing then it is also a sign of overfitting.
6. **How does Dropout reduce overfitting?**
Dropout randomly removes some neurons during training. This stops the model from depending on only some neurons and helps it learn better features. It can improve the model performance on new data.
7. **What is the purpose of Batch Normalization?**
Batch Normalization normalizes the activations in a mini-batch. It helps the network to train more stable and can make convergence faster. It also helps in reducing some training problems.
8. **Explain the numerical Batch Normalization example.**
For the values $[2, 4, 6, 8]$ the mean is 5 and the variance is 5. After normalization the values become approximately $[-1.342, -0.447, 0.447, 1.342]$. This makes the values close to zero-centred and properly scaled.
9. **What are the roles of γ and β ?**
 γ is used to change the scale of the normalized value and β is used to shift the value. They are learnable parameters of Batch Normalization. The network learns their values during training.
10. **Compare SGD, Momentum, RMSProp and Adam.**
SGD updates the weights using the gradient. Momentum uses previous updates to make learning faster and smoother. RMSProp changes the learning rate based on the gradients. Adam combines ideas from Momentum and RMSProp and usually gives fast convergence.
11. **What happens when the learning rate is too large?**
If the learning rate is too large then the model can make very big updates. The loss may jump up and down and the model may not converge. It can also give poor accuracy.
12. **What happens when the learning rate is too small?**
If the learning rate is too small then the weights change very slowly. The model takes more time to learn and training can take many epochs. It may also get stuck before reaching good performance.
13. **What is the effect of increasing batch size?**
Increasing batch size means more training samples are used for one weight update. The gradient becomes more stable but it can require more memory. A very large batch size may also give less variation during training.
14. **Explain stride and padding.**
Stride tells how many pixels the filter moves after each step. A larger stride makes the output feature map smaller. Padding means adding extra pixels around the input. It helps to control the output size and keep edge information.
15. **Why is MobileNetV2 computationally efficient?**
MobileNetV2 uses depthwise convolution and pointwise 1×1 convolution. These operations need less computation and fewer parameters than normal convolution. It is useful for systems where computing power is limited.
16. **What is depthwise separable convolution?**
Depthwise separable convolution divides normal convolution into two operations. First depthwise convolution applies a filter to each channel separately. Then pointwise convolution combines the channels. This reduces the number of calculations.
17. **What is transfer learning?**
Transfer learning means using a model that is already trained on a large dataset for a new task. For example MobileNetV2 trained on ImageNet can be used for the Oxford-IIIT Pet dataset. This saves training time and the model already has useful features.
18. **Differentiate feature extraction and fine-tuning.**
In feature extraction the pretrained layers are frozen and only the new classifier is trained. In fine-tuning some

pretrained layers are unfrozen and trained again. Feature extraction is faster but fine-tuning can help the model adapt better to the new dataset.

19. Why is a smaller learning rate generally used during fine-tuning?

The pretrained model already has useful features. A large learning rate can change these features too much and reduce the learned knowledge. A small learning rate makes small changes and helps the model adjust to the new dataset.

20. Why is K-Fold Cross-Validation useful for hyperparameter selection?

K-Fold Cross-Validation tests a model on different parts of the training data. It gives a better idea about how stable the model performance is. It helps us choose a configuration that works well on different folds.

21. Why must the test set remain untouched during tuning?

The test set should only be used for the final evaluation. If we use it during tuning then the model selection can become biased. The final test accuracy will not give a fair idea of the model performance.

22. Why should mean and standard deviation both be reported?

Mean shows the average performance of the model across the folds. Standard deviation shows how much the results change between the folds. A low standard deviation means the model performance is more stable.

23. Is the highest validation accuracy always sufficient to select a model?

No. Highest validation accuracy is not the only thing we should check. We should also check standard deviation, test accuracy and training time. A model with slightly lower accuracy but better stability can be a better choice.

16. Additional Exercise

Select two new combinations of learning rate, dropout, batch size and fine-tuning strategy. Evaluate them using 5-fold cross-validation and compare their results with the selected configuration.

Students must justify whether the new configuration is preferable based on accuracy, standard deviation, computational cost and final test performance.

17. Expected Outcome

Students should experimentally demonstrate that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. They should understand the MobileNetV2 architecture, perform transfer learning and fine-tuning, and select a reliable final configuration using 5-fold cross-validation.

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>

19. GitHub Repository

[https://github.com/Nitheesha-1025/Deep-Learning/tree/CNN-training\(finetuning%2C-transfer-learning\)](https://github.com/Nitheesha-1025/Deep-Learning/tree/CNN-training(finetuning%2C-transfer-learning))